CIFAKE: IMAGE CLASSIFICATION AND EXPLAINABLE SYNTHETIC IMAGE IDENTIFICATION

Computer Vision Module - Deep Learning

Project: CIFAKE Deep Learning Detection Engine

Focus Group: AI Development Team

CHAPTER 1: INTRODUCTION

Project Background With the rapid proliferation of highly capable generative AI models (such as Midjourney, Stable Diffusion, and DALL-E), the internet is experiencing an influx of hyper-realistic synthetic media. Distinguishing between authentic, camera-captured photographs and AI-generated images has become increasingly difficult for the human eye, raising concerns regarding misinformation, digital forgery, and copyright infringement.

Problem Statement There is a critical need for reliable, automated tools to verify the authenticity of digital images. Current manual verification methods are slow and subjective, and many existing deep learning solutions act as "black boxes," providing predictions without any context or explanation as to why an image was flagged as fake.

Objectives 1. To develop a robust Deep Learning classifier capable of distinguishing between REAL and AI-GENERATED images with high accuracy. 2. To implement an Explainable AI (XAI) module that visually explains the model's decision-making process. 3. To design and deploy a professional, user-friendly web application for real-time image analysis.

Scope of the Project The project encompasses the training and deployment of a MobileNetV2-based model on the CIFAKE dataset (CIFAR-10 images vs. Stable Diffusion generated fakes). It includes the development of a Streamlit frontend and the integration of Grad-CAM for explainability. The scope is limited to binary classification (Real vs. Fake) on standard image formats.

Expected Outcomes A fully functional, web-based AI application that accepts image uploads, accurately predicts their authenticity, displays confidence metrics, and provides a Grad-CAM heatmap visualization explaining the prediction.

CHAPTER 2: SYSTEM OVERVIEW

Existing System Existing systems for synthetic image detection often rely on either basic metadata analysis (which can be easily stripped or forged) or complex, unexplainable deep neural networks. These "black box" systems output a simple probability score, leaving users guessing as to what triggered the detection, leading to a lack of trust in the system's output.

Proposed System The proposed system, CIFAKE, bridges the gap by combining a highly optimized Transfer Learning model (MobileNetV2) with a Gradient-weighted Class Activation Mapping (Grad-CAM) module. This ensures that every prediction is accompanied by a visual heatmap, explicitly showing the regions (e.g., specific artifacts, blending errors) that influenced the model's decision.

System Modules 1. **Inference Engine Module:** Handles image loading, preprocessing, and execution of the TensorFlow/Keras model. 2. **Explainability (XAI) Module:** Computes spatial gradients from the final convolutional layer to generate transparent heatmaps. 3. **Frontend Application Module:** A Streamlit-based graphical user interface for seamless user interaction and result visualization. 4. **Analytics Module:** Displays static performance metrics and dataset distributions.

Overall Workflow The user uploads an image via the web interface. The image is passed to the Inference Engine, which resizes and normalizes the data before feeding it through the MobileNetV2 model to obtain a probability score. Concurrently, the Explainability Module extracts feature maps from the model to generate a Grad-CAM overlay. The final results, including the original image, prediction label, confidence score, and visual explanation, are displayed to the user.

CHAPTER 3: LITERATURE SURVEY

Research Papers Reviewed 1. *Deep Residual Learning for Image Recognition* (He et al.) - Foundation for deep CNN architectures. 2. *MobileNetV2: Inverted Residuals and Linear Bottlenecks* (Sandler et al.) - Explored for its efficiency and low latency in vision tasks. 3. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization* (Selvaraju et al.) - The theoretical basis for the project's explainability module.

Existing Solutions Current commercial solutions like Truepic or specific academic ensemble models focus heavily on frequency-domain analysis or massive Vision Transformers. While highly accurate, they are often computationally expensive and lack granular visual explainability.

Comparative Analysis Compared to massive Vision Transformers, MobileNetV2 requires significantly fewer parameters (approx. 3.4 million), making inference nearly instantaneous on standard CPUs. Furthermore, the explicit integration of Grad-CAM in CIFAKE provides a layer of transparency not commonly found in standard

commercial APIs.

Research Gap There is a noticeable gap in lightweight, web-deployable synthetic image detectors that prioritize both speed and explainability. CIFAKE directly addresses this by providing a highly optimized model with a built-in XAI interface.

CHAPTER 4: TECHNOLOGIES USED

Programming Languages * Python 3.9+ (Core backend and scripting) * CSS3 (Custom frontend styling)

Frameworks and Libraries * **TensorFlow & Keras:** For model loading, inference, and gradient tape operations. * **Streamlit:** For rapid development of the professional web frontend. * **OpenCV (cv2):** For colormap application and image manipulation in the Grad-CAM module. * **NumPy:** For high-performance numerical operations and array manipulations. * **Pillow (PIL):** For initial image loading, resizing, and format conversion.

Development Tools * Git & GitHub (Version control) * VS Code / Advanced AI Editor (Development environment)

CHAPTER 5: ARCHITECTURE AND DESIGN

System Architecture Diagram *(Note: A conceptual diagram)* [User Uploads Image] ---> [Streamlit Frontend] ---> [Preprocessing (PIL/NumPy)] | v [Grad-CAM Explanation] <--- [MobileNetV2 Model] <-------+ | | v v [Frontend Renders Heatmap] [Frontend Renders Prediction & Confidence]

Workflow Explanation The architecture follows a modular, feed-forward design. The frontend is strictly decoupled from the model inference logic. `app.py` and the `pages/` directory handle routing and state management (such as prediction history). The `model/` directory encapsulates `predict.py` (implementing a singleton pattern to keep the model in memory) and `gradcam.py` (which hooks into the `global_average_pooling2d` layer's inputs to extract gradients without breaking the nested functional graph).

CHAPTER 6: IMPLEMENTATION

Key Features Implemented 1. **Dynamic Model Loading:** A singleton pattern ensures `cifake_model.keras` is loaded only once, drastically reducing prediction latency. 2. **Robust Error Handling:** The system gracefully intercepts corrupted files, extreme resolutions, and non-image formats without crashing. 3. **Grad-CAM Integration:** Successfully extracts feature maps from nested Keras models to overlay jet-colormapped heatmaps on original inputs. 4. **Session History:** Implemented Streamlit Session State to persist user predictions across the session lifecycle. 5. **Professional UI:** Injected custom CSS to transform default Streamlit into a modern, dark-themed analytical dashboard.

CHAPTER 7: RESULTS AND DISCUSSION

Screenshots *(Refer to the application's Detection and Explainability pages for visual results).*

Output Analysis The model successfully outputs binary classification (REAL vs. AI GENERATED) accompanied by a confidence percentage derived from the sigmoid output probability.

Performance Evaluation The model achieved the following final metrics on the 20,000-image test set: * **Validation Accuracy:** 93.32% * **Validation Loss:** 0.1690

Testing Results The application passed all edge-case tests, including empty uploads, extremely large images (handled safely via PIL), and out-of-distribution artifacts. As noted during testing, out-of-distribution images (e.g., text, indoor scenes) trigger deterministic but sometimes incorrect classifications, a known limitation of training strictly on CIFAR-10 derived datasets.

CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

Challenges Faced 1. **Nested Model Gradients:** Keras 3 strictly partitions graphs. Attempting to extract the output of the nested `mobilenetv2_1.00_224` base model for Grad-CAM threw graph disconnection errors

(`Output with path 0 is not connected to inputs`). 2. **Out-of-Distribution Data:** The model struggled to classify images heavily featuring text or objects not present in the CIFAKE (CIFAR-10) dataset.

Solutions Implemented 1. **Architectural Hooking:** Instead of probing inside the nested base model, the Grad-CAM script was refactored to hook into the *input* of the immediately following layer (`global_average_pooling2d`), effectively extracting the exact required feature map while maintaining graph integrity. 2. **UI Disclaimers:** Added documentation and analytics notes to set proper user expectations regarding the model's operational domain.

Technical Skills Learned * Advanced TensorFlow graph manipulation and GradientTape usage. * Streamlit multi-page application routing and state management. * UI/UX optimization using custom CSS within Python web frameworks.

CHAPTER 9: FUTURE ENHANCEMENTS

Proposed Improvements * **Ensemble Modeling:** Combining MobileNetV2 with an EfficientNet variant to improve generalization on out-of-distribution data. * **Dataset Expansion:** Retraining the model on a broader dataset containing human faces, natural landscapes, and text to reduce false positives on everyday photographs.

Additional Features * **Batch Processing:** Allowing users to upload a ZIP file of images for bulk verification. * **Report Generation:** Adding a feature to export the prediction and Grad-CAM heatmap as a downloadable PDF certificate.

Scalability Opportunities The inference logic can easily be abstracted into a FastAPI REST endpoint, allowing the model to serve mobile applications or browser extensions at scale using cloud platforms like Google Cloud Run or AWS ECS.

CHAPTER 10: CONCLUSION

Summary of Work Completed The CIFAKE project successfully transitioned from a raw, trained `.keras` model into a production-ready web application. It features a robust prediction pipeline, an integrated Explainable AI module using Grad-CAM, and a highly professional user interface.

Achievements * Maintained a strict 93.32% validation accuracy while adding significant operational overhead. * Delivered a visually striking, user-friendly application without relying on external databases or complex backends. * Successfully mitigated complex nested-model gradient extraction issues.

Project Impact This project provides a transparent, educational, and effective tool for identifying synthetic media. By prioritizing explainability, it helps demystify "black box" AI decisions, fostering trust and critical analysis among users interacting with digital media.

Final Remark As generative AI continues to evolve, tools like CIFAKE serve as a critical first line of defense in preserving digital authenticity. The foundations laid in this project can easily be expanded to tackle the deepfakes of tomorrow.